EdgeAI ControlLab: On-Device System Identification, Neural Self-Tuning and Anomaly Detection for DC Motor Speed Control

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology
School of Electronics Engineering (SENSE)

by

Pratik Kumar (24BML0027)

Thakur Chand Choudhary (24BML0012)

Under the guidance of

Dr. Venugopal P

Professor Grade 1, SENSE



October 2026

Executive Summary

Speed control of small DC motors is still dominated by PID controllers whose gains are tuned by hand for one motor, one load and one supply. Recent AI tuners improve on this but rely on reinforcement learning or large networks that must be trained on every machine, and machine-learning fault detectors need labelled examples of every fault. EdgeAI ControlLab moves the intelligence onto a low-cost ESP32-S3 microcontroller that drives a JGA12-N20 gear motor at 100 Hz. A bank of recursive-least-squares estimators identifies the motor's gain, time constant and dead time from live data; a 370-parameter neural network, trained offline on 4,000 simulated motors, converts that model into PI gains; and a digital-twin anomaly detector learns a healthy baseline and flags departures without any fault labels. A FastAPI backend and a React dashboard with a 3D digital twin provide live monitoring, analysis and reporting. On the bench motor the learned model reproduced the measured speed with a 96.6 % fit, the neural tuner settled speed steps in 0.15–0.27 s against 0.53–0.73 s for the analytic lambda rule with at most 1.3 % overshoot, and stress testing produced no false anomaly alarms once the vibration baseline was made speed-dependent. The firmware uses 6.3 % of the controller's RAM and 10.4 % of its flash, and the complete rig costs about ₹3,100.

CONTENTS

Page No.

Executive Summary ..... **ii**

LIST OF FIGURES ..... **iv**

LIST OF TABLES ..... **v**

LIST OF ABBREVIATIONS ..... **v**

1. INTRODUCTION **1**

1.1 Literature Review 1

1.1.1 Classical tuning and system identification 1

1.1.2 AI-assisted PID tuning (2024–2025) 2

1.1.3 Fault and anomaly detection at the edge 3

1.2 Research Gap 3

1.3 Problem Statement 4

1.3.1 Relevance of the problem statement w.r.t. SDG 4

1.4 Organisation of the Report 5

2. PROJECT OBJECTIVE **6**

3. PROPOSED WORK **7**

3.1 Design Approach / System Model / Algorithm 7

3.1.1 System architecture 7

3.1.2 Plant model 7

3.1.3 Speed controller 8

3.1.4 On-device identification with a recursive-least-squares bank 9

3.1.5 Neural PI tuner 9

3.1.6 Anomaly detection with a digital twin 11

3.1.7 Safety layer 12

3.2 Technical Descriptions 13

3.2.1 Hardware setup 13

3.2.2 Firmware architecture 14

3.2.3 Communication protocol 15

3.2.4 Backend server 16

3.2.5 Laboratory dashboard and 3D digital twin 16

3.2.6 Virtual sensors 17

4. HARDWARE/SOFTWARE TOOLS USED **19**

5. RESULT ANALYSIS **20**

5.1 Hardware Bring-up and Open-loop Characterisation 20

5.2 System Identification Results 20

5.3 Neural Tuner: Offline Evaluation 22

5.4 Closed-loop Performance 23

5.5 Frequency-domain Verification 24

5.6 Anomaly Detection 25

5.7 Computational Footprint 26

5.8 Discussion 27

6. CONCLUSION AND FUTURE WORK **28**

6.1 Summary 28

6.2 Limitations and Constraints 28

6.3 Improvement / Future Work 28

7. SOCIAL AND ENVIRONMENTAL IMPACT **29**

8. WORK PLAN **30**

8.1 Timeline 30

8.2 Individual Contribution 30

9. COST ANALYSIS **31**

10. REFERENCES **32**

Appendix-1: Firmware Source Code (Edge-AI Modules) **34**

LIST OF FIGURES

Figure 3.1: Three-layer architecture of EdgeAI ControlLab 7
Figure 3.2: Adaptive speed loop: PI controller, on-device identification, neural tuner and digital-twin anomaly detector 8
Figure 3.3: On-device identification and confidence gating 10
Figure 3.4: Offline training pipeline of the neural PI tuner 11
Figure 3.5: Anomaly classification from z-scores, drift and thermal margin 12
Figure 3.6: Photograph of the bench rig with the main components labelled; the inset shows the DS18B20 and MPU6050 mounted on the side of the motor 13
Figure 3.7: Wiring of the ESP32-S3 to the driver, sensors and motor 14
Figure 3.8: Sequence executed by the 100 Hz control task in every period 15
Figure 3.9: Dashboard overview: setpoint, speed and PWM during step excitation in Adaptive AI mode 17
Figure 3.10: Digital Twin 3D tab: rendered rig driven by telemetry, with speed gauge and live readouts. 17
Figure 3.11: AI Predictor tab: neural-tuner gains, lambda-rule gains and model card with feature importance 18
Figure 3.12: Thermal model used as a virtual temperature sensor: 10 min at 150 RPM followed by cooling 18
Figure 5.1: Logged hardware response to 50 ↔ 150 RPM steps in Adaptive AI mode: speed (top) and PWM (bottom) 20
Figure 5.2: Delay bank on bench data: one-step prediction error (left) and free-run fit (right); d = 3 is best on both 21
Figure 5.3: Measured speed versus free-run simulation of the identified models with d = 3 and d = 1 .21
Figure 5.4: Training data: identified-delay distribution (left) and permutation feature importance (right) 22
Figure 5.5: Closed-loop cost relative to the optimum on 300 unseen simulated motors 22
Figure 5.6: Neural-tuner gain maps for d = 3 (dot: bench motor, K = 1.17, τ = 73 ms) 23
Figure 5.7: Simulated step sequence on a motor matched to the bench identification 24
Figure 5.8: Bode plot of the identified plant and open loop, with phase and gain margins 25
Figure 5.9: Measured vibration versus speed on the bench motor, with 40 RPM band means 25
Figure 5.10: First hardware stress run with the linear vibration baseline: speed, vibration z-score and reported class 26
Figure 8.1: Project timeline (blue: completed; orange: remaining) 30

LIST OF TABLES

Table 1.1: Summary of recent literature (2024–2025) 3
Table 2.1: Measurable targets 6
Table 3.1: Hard protection limits in the firmware 12
Table 3.2: ESP32-S3 pin allocation 14
Table 3.3: Main telemetry fields (one JSON object per 10 ms sample) 16
Table 4.1: Hardware components 19
Table 4.2: Software tools 19
Table 5.1: Identification of the bench motor for each candidate delay (264 informative samples) 21
Table 5.2: Generalisation of the tuners (cost relative to grid-search optimum) 23
Table 5.3: Real-motor comparison of the two adaptive gain sources 24
Table 5.4: Resource use on the ESP32-S3 26
Table 8.1: Individual contributions 30
Table 9.1: Bill of materials and cost 31

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CDC	Communications Device Class (USB serial)
DC	Direct Current
DDPG	Deep Deterministic Policy Gradient
DQN	Deep Q-Network
FOPDT	First-Order Plus Dead Time
FOPID	Fractional-Order PID
GAN	Generative Adversarial Network
GM / PM	Gain Margin / Phase Margin
I²C	Inter-Integrated Circuit bus
IMC	Internal Model Control
IMU	Inertial Measurement Unit
LEDC	LED Control (ESP32 PWM peripheral)
MLP	Multi-Layer Perceptron
OLED	Organic Light-Emitting Diode display
PCNT	Pulse Counter peripheral
PI / PID	Proportional–Integral / Proportional–Integral–Derivative
PSO	Particle Swarm Optimisation
PWM	Pulse-Width Modulation
REST	Representational State Transfer
RL	Reinforcement Learning
RLS	Recursive Least Squares
RMS	Root Mean Square
RPM	Revolutions Per Minute
SDG	Sustainable Development Goal
TD3	Twin Delayed Deep Deterministic policy gradient
TinyML	Machine learning on microcontrollers

1. INTRODUCTION

Electric motors are the workhorses of modern engineering. Motor-driven systems account for roughly 45 % of global electricity consumption [1], and small permanent-magnet DC gear motors in particular are everywhere: robots, drones, medical pumps, camera gimbals, conveyors, automotive actuators and laboratory equipment. In almost all of these the speed or position loop is closed by a proportional–integral–derivative (PID) controller, because it is simple, well understood and cheap to compute [2].

The weakness of PID control is not the algorithm but its tuning. The gains that give a crisp, well-damped response depend on the motor's gain, its mechanical time constant and the dead time in the drive and the measurement. These change from one motor to the next, with load, with supply voltage and with temperature and wear. In practice gains are set by trial and error or by classical rules such as Ziegler–Nichols [3], and are then left alone. Our own first experiment illustrates the problem: with a generic gain set (Kp = 1.25, Ki = 0.45, Kd = 0.05) the bench motor settled at 123 RPM when asked for 150 RPM, an error of 18 %.

At the same time, microcontrollers have become powerful enough to run small machine-learning models in real time. This field, usually called TinyML or Edge AI, brings inference to the device instead of the cloud, with low latency, no network dependency and very low cost [4], [5]. A dual-core ESP32-S3 running at 240 MHz has ample capacity to identify a motor, compute its controller gains and watch its health, all inside the control loop.

This project, **EdgeAI ControlLab**, builds a complete laboratory platform around that idea. An ESP32-S3 controls a JGA12-N20 encoder gear motor at 100 Hz and, while doing so, (i) learns a model of the motor from live data, (ii) converts the model into PI gains with a small neural network that was trained offline on thousands of simulated motors, and (iii) detects abnormal behaviour against a healthy baseline that it learns itself, using a digital-twin copy of the learned model. A FastAPI backend and a React dashboard with a 3D digital twin make every internal quantity visible in real time and turn the rig into a teaching and research tool for control engineering.

1.1 Literature Review

1.1.1 Classical tuning and system identification

Model-free tuning rules began with Ziegler and Nichols, who set gains from the ultimate gain

and period of a loop brought to the edge of oscillation [3]. Their responses are aggressive and the experiment is hard on hardware. Model-based rules replaced them: internal model control (IMC) gives PI gains in closed form from a first-order-plus-dead-time (FOPDT) model, with a single closed-loop time constant λ as the tuning knob [6]. Skogestad's SIMC rules refined the choice of λ and the treatment of dead time [7], and O'Dwyer catalogues several hundred such rules [8]. All of them assume that the model is known.

System identification supplies that model from input–output data [9]. For embedded use, recursive least squares (RLS) is attractive because it updates the parameter estimate sample by sample with a fixed, small amount of computation [10], [11]. Combining a recursive estimator with a tuning rule gives the classical self-tuning regulator [12]. Its known practical difficulties are estimator wind-up when the input is not exciting and drift of the estimate under weak excitation; forgetting-factor management and data gating are the usual remedies [13]. These difficulties shaped several design decisions in this project.

1.1.2 AI-assisted PID tuning (2024–2025)

Recent research has moved towards learning-based tuners. Sahrir and Basri trained a Levenberg–Marquardt feed-forward network, initialised from particle-swarm-optimised gains, to output PID gains for a quadcopter's attitude and altitude loops, and reported lower overshoot and settling time in simulation [14]. Zhou et al. proposed a GAN-based PID (G-PID) in which a generator produces compensating actions so that an unknown or changing pneumatic-muscle joint behaves like a user-chosen reference system; it was validated on prosthesis prototypes with time-varying payloads [15].

Reinforcement learning (RL) [16] is the most active direction. Bujgoi and Sendrescu used the TD3 actor–critic algorithm to tune PID controllers for a strongly nonlinear bioprocess and compared it with classical tuning in simulation [17]. Hao et al. combined a DDPG agent, which adjusts PID parameters online from the tracking error and system state, with an auxiliary servo valve to improve position control of a hydraulic injection-moulding servo [18]. Shahouni et al. tuned a fractional-order PID for a flow controller with time delay in a desalination plant using a hybrid of particle swarm optimisation and a deep Q-network, outperforming GA, fuzzy and NN-PID baselines but at a noted computational cost [19].

These studies show that learned tuners can beat fixed rules, but they share three traits that limit their use on small embedded controllers: they are trained through many interactions with the

specific plant (or its accurate simulation), the trained policies are large and usually run on a PC, and the delay in the loop is either ignored or handled by heavy optimisation.

1.1.3 Fault and anomaly detection at the edge

Model-based fault detection compares measured behaviour with a model and analyses the residuals [20]. Data-driven alternatives treat the problem as anomaly detection, flagging data that do not conform to learned normal behaviour [21], or as supervised fault classification, which has been reviewed extensively for rotating machinery [22]. Predictive maintenance built on such monitoring reduces downtime and extends equipment life [23]. The digital twin, a live virtual counterpart of a physical asset, has become a common frame for this work [24]; Castellani et al. used digital-twin simulations to generate normal data for anomaly detection when real faults are scarce [25].

At the edge, Ayadi et al. (2025) trained a multi-layer-perceptron classifier on IMU features from a DC-motor power-transmission rig with eleven operating conditions and deployed it on a Cortex-M4F microcontroller. Accuracy was 97.1 % in development but 73.19 % on unseen data on the device, and every fault class had to be produced and labelled in advance [26]. Benchmarks such as MLPerf Tiny confirm that inference of this size fits comfortably on microcontrollers [27]; the harder problem is obtaining representative training data.

Table 1.1 summarises the most relevant recent studies and what each implies for this work.

Table 1.1: Summary of recent literature (2024–2025)

Study	Method	Application	Implication for this work
Sahrir & Basri, 2024 [14]	NN outputs PID gains (PSO-initialised)	Quadcopter UAV	A network can map conditions to gains
Zhou et al., 2024 [15]	GAN compensator + PID	Pneumatic-muscle joint	Adaptation to unknown or changing plants
Bujgoi & Sendrescu, 2025 [17]	TD3 reinforcement learning	Nonlinear bioprocess	RL needs many episodes per plant
Hao et al., 2025 [18]	DDPG-adjusted PID	Hydraulic servo	Online gain adjustment by deep RL
Shahouni et al., 2025 [19]	PSO + DQN, FOPID	Flow control with delay	Delay matters; heavy optimisation
Ayadi et al., 2025 [26]	TinyML MLP classifier	DC-motor transmission faults	97.1 % offline, 73.2 % on device; needs fault labels

1.2 Research Gap

The review points to a clear gap between what learned tuners and detectors achieve in

simulation or on PCs and what a low-cost motor controller can do on its own:

- **Tuning is plant-specific.** RL and GAN tuners learn through interaction with each plant. A new motor, or the same motor with a new load, requires retraining. None of the reviewed methods learns a general mapping from a plant model to gains that can be reused on any motor.
- **Learning rarely happens on the controller.** Identification and tuning are usually performed offline or on a PC. Few works identify the plant on the microcontroller itself, inside the real-time loop, and decide when the identified model is trustworthy enough to act on.
- **Dead time is under-treated.** The delay introduced by the driver, the speed estimator and the sampling is often ignored, yet it is what limits how aggressive the gains can be.
- **Fault detectors need labelled faults.** Supervised edge classifiers require every fault to be produced and labelled beforehand, and their accuracy falls on unseen data. An unsupervised detector that learns normal behaviour on the machine itself removes this burden.
- **Laboratory access.** Commercial control-teaching rigs are expensive, so students rarely see identification, tuning and fault detection working together on real hardware.

1.3 Problem Statement

To design and build a low-cost motor-control platform in which an ESP32-S3 microcontroller, without a PC in the loop, (a) controls the speed of a DC gear motor in real time, (b) identifies the motor's gain, time constant and dead time from live operating data and decides when that model can be trusted, (c) converts the model into PI gains using a neural network trained once, offline, on a wide population of simulated motors, and (d) detects abnormal operation by comparison with a healthy baseline and a digital-twin model that it learns itself, without fault labels, while a host dashboard makes every step observable for teaching and analysis.

1.3.1 Relevance of the problem statement w.r.t. SDG

- **SDG 9 – Industry, Innovation and Infrastructure:** self-tuning, self-monitoring motor drives built from commodity parts lower the barrier to automation for small manufacturers and support resilient infrastructure.
- **SDG 7 – Affordable and Clean Energy:** motors consume a large share of electricity [1];

well-tuned loops avoid oscillation and wasted control effort, and early detection of extra load or friction prevents inefficient operation.

- **SDG 4 – Quality Education:** a rig that costs about ₹3,100 makes identification, adaptive control and condition monitoring available to every student laboratory, not only to well-funded ones.
- **SDG 12 – Responsible Consumption and Production:** condition monitoring extends equipment life and enables maintenance on need rather than on failure or fixed schedule [23].

1.4 Organisation of the Report

Chapter 2 states the project objectives and measurable targets. Chapter 3 presents the proposed work: the system architecture, plant model and control law, the three Edge-AI algorithms (on-device identification, the neural PI tuner and digital-twin anomaly detection), and the hardware, firmware, communication, backend and dashboard that implement them. Chapter 4 lists the hardware and software tools. Chapter 5 analyses the results obtained on the bench motor and in simulation: identification accuracy, tuner generalisation, closed-loop and frequency-domain performance, anomaly detection and the computational footprint. Chapter 6 concludes with limitations and future work, Chapter 7 discusses the social and environmental impact, and Chapters 8 and 9 give the work plan and cost analysis. The references and an appendix with the Edge-AI firmware listings complete the report.

2. PROJECT OBJECTIVE

The aim of the project is a motor controller that learns, tunes and monitors itself, and a laboratory platform that exposes all of it. The specific objectives are:

1. **Real-time control.** Close a 100 Hz speed loop on a JGA12-N20 gear motor with an ESP32-S3, with current, vibration and temperature monitoring and hard safety limits (over-current, stall, over-temperature, host-link loss).
2. **On-device identification.** Estimate the motor's steady-state gain K, time constant τ and dead time d from live data in the control task, and gate the result with an explicit confidence test.
3. **Neural self-tuning.** Train a PI tuner offline on simulated motors and run it on the ESP32 so that an identified model is turned into gains within seconds (Adaptive AI mode), with bumpless switching.
4. **Label-free anomaly detection.** Score the motor's behaviour against a learned healthy baseline and a digital-twin model and classify departures into interpretable classes, without any labelled fault data.
5. **Laboratory dashboard.** Provide live telemetry, identification, frequency and state-space analysis, a 3D digital twin, saved runs and exportable reports, with a simulator mode so the software can be used without hardware.

Measurable targets were set for each objective and are evaluated in Chapter 5:

Table 2.1: Measurable targets

Objective	Target	Achieved (Chapter 5)
Control loop	100 Hz, jitter-tolerant speed estimate	100 Hz on core 1; speed from measured elapsed time
Identification	Free-run fit > 90 % on bench data	96.6 % (d = 3 samples)
Neural tuner	Faster than lambda rule, overshoot ≤ 5 %	3.5× faster settling, ≤ 1.3 % overshoot
Generalisation	Near-optimal on unseen motors	Median cost 1.016× optimum on 300 new motors
Anomaly detection	No false alarms in normal operation	0 after per-speed-band baseline
Footprint	< 25 % of RAM and flash	6.3 % RAM, 10.4 % flash
Cost	< ₹5,000 for the rig	≈ ₹3,100

3. PROPOSED WORK

This chapter describes the design: the system architecture, the plant model, the control law, the three Edge-AI algorithms that run on the microcontroller, and the hardware and software that implement them.

3.1 Design Approach / System Model / Algorithm

3.1.1 System architecture

The platform has three layers (Figure 3.1). The **physical rig** contains the motor with its quadrature encoder, a TB6612FNG H-bridge, an INA219 current sensor, an MPU6050 accelerometer, an SSD1306 OLED and the supply. The **edge layer** is the ESP32-S3, which does all control, learning and monitoring. The **host layer** is optional: a FastAPI backend bridges the serial link to the browser and a React dashboard visualises the data. The guiding principle is that learning runs at the edge and the PC only watches; if the host disconnects, a watchdog stops the motor rather than letting it run unattended.

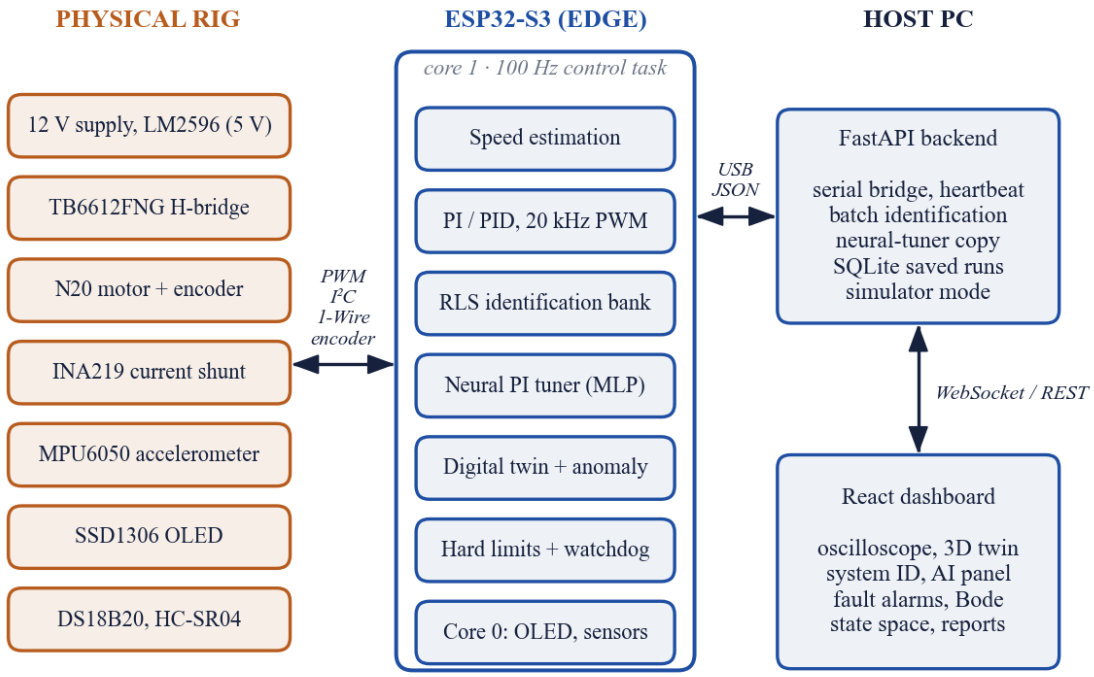


Figure 3.1: Three-layer architecture of EdgeAI ControlLab

3.1.2 Plant model

A DC gear motor driven through a PWM H-bridge behaves, for speed control, approximately as a first-order system with dead time. With PWM duty u (0–255) as input and output-shaft

speed y (RPM), the continuous model is

$$G(s) = Y(s)/U(s) = K \cdot e^{(-\theta s)} / (\tau s + 1) \qquad (3.1)$$

where K is the steady-state gain in RPM per PWM step, τ the mechanical time constant and θ the dead time. Sampling at Ts = 10 ms and adding an offset c for friction and dead-zone gives the discrete model that the controller identifies:

$$y[k] = a \cdot y[k-1] + b \cdot u[k-d] + c, \quad a = e^{(-Ts/\tau)}, \quad K = b/(1-a) \qquad (3.2)$$

The delay d (in samples) collects the driver delay, the motor's electrical lag and, importantly, the speed estimator: speed is computed from the encoder count difference over a five-sample (50 ms) window, which by itself adds about two samples of effective delay. Treating d as an unknown to be identified, rather than assuming d = 1, turned out to be essential (Section 5.2).

3.1.3 Speed controller

The speed loop (Figure 3.2) uses a PI law with derivative on measurement (the D term is available in manual PID mode but disabled in Adaptive AI mode):

$$u[k] = Kp \cdot e[k] + Ki \cdot \Sigma\, e[j] \cdot Ts - Kd \cdot (y[k] - y[k-1])/Ts \qquad (3.3)$$

Three refinements make it robust on real hardware. **Conditional integration** stops the integrator from winding up while the PWM is saturated in the direction of the error. **Bumpless transfer** rescales the integral whenever Ki changes, $I \leftarrow I \cdot Ki,old/Ki,new$, so that the integral contribution to the PWM stays continuous when the tuner updates the gains; without it a gain switch caused a 40 % overshoot in testing. **Timestamped speed estimation** divides the count difference by the measured elapsed time, so a control cycle delayed by I²C traffic does not appear as a speed spike.

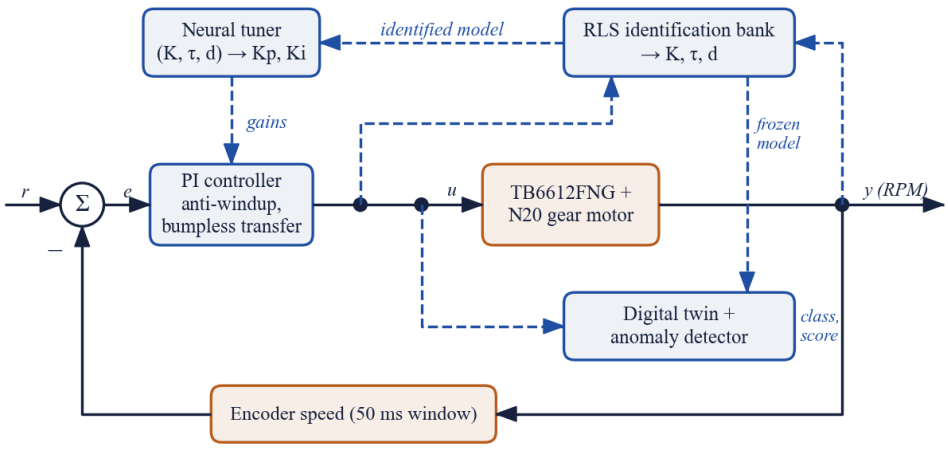


Figure 3.2: Adaptive speed loop: PI controller, on-device identification, neural tuner and digital-twin anomaly detector

3.1.4 On-device identification with a recursive-least-squares bank

For each candidate delay d = 1, …, 6 the firmware runs an independent three-parameter RLS estimator for $\theta = [a, b, c]^T$ with regressor $\varphi[k] = [y[k-1], u[k-d], 1]^T$ (signals normalised by 300 RPM and 255 PWM to keep single-precision arithmetic well conditioned). Each estimator performs the standard update [10], [11]:

$$\varepsilon = y[k] - \varphi^T\theta, \quad g = P\varphi/(\lambda + \varphi^T P\varphi), \quad \theta \leftarrow \theta + g\cdot\varepsilon, \quad P \leftarrow (P - g\varphi^T P)/\lambda \tag{3.4}$$

with forgetting factor $\lambda = 0.995$ (an effective memory of about 200 informative samples). Four safeguards address the known failure modes of recursive estimators [13]:

- **Informative-sample gating.** Estimators update only when the speed is changing ($|\Delta y| > 0.6$ RPM) or the PWM changed within the last seven samples. In steady state the regressor is constant and forgetting would otherwise inflate P (covariance wind-up). The gate is shared, so all six delay models compete on identical data.
- **Covariance limiting and pole clamping.** The trace of P is capped at 1000 and a is clamped to the physically plausible range 0.30–0.995.
- **Delay selection with hysteresis.** Each estimator tracks its one-step prediction error; the active delay changes only when another model is at least 10 % better.
- **Confidence gate.** The model is used only after more than 60 informative updates, with K and τ plausible, settled (within 5 % and 15 % of their own slow averages) and with at least 40 PWM steps of input range in the data. Learning is suspended while a D term is active, because derivative action injects measurement noise into the input and was found to bias K.

The resulting algorithm is shown in Figure 3.3. From the selected model, $K = b/(1-a)$ and $\tau = -Ts/\ln a$.

3.1.5 Neural PI tuner

The baseline for turning the identified model into gains is the IMC or lambda rule [6], with desired closed-loop time constant $\lambda c = 0.15$ s:

$$Kp = \tau / (K\cdot(\lambda c + d\cdot Ts)), \quad Ki = 1 / (K\cdot(\lambda c + d\cdot Ts)) \tag{3.5}$$

The rule is safe but conservative, and it is only as good as the FOPDT approximation. The neural tuner instead learns the mapping $(K, \tau, d) \rightarrow (Kp, Ki)$ that minimises a closed-loop cost. Its training pipeline (Figure 3.4) is:

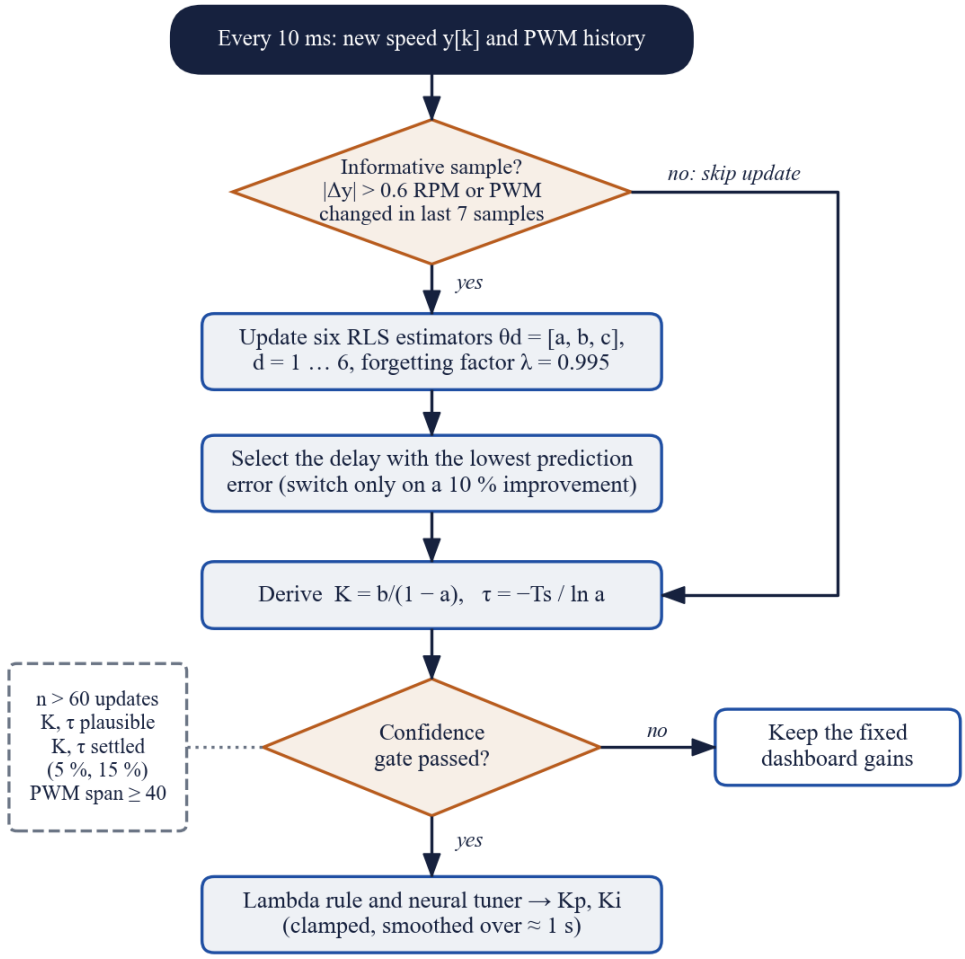


Figure 3.3: On-device identification and confidence gating

1. Sample 4,000 random motors with K from 0.3 to 3 RPM/PWM, τ from 20 to 600 ms, true delay 1–5 samples, a PWM dead-zone of 0–15 and speed noise of 0.1–0.6 RPM (log-uniform where appropriate).
2. Simulate each motor exactly as the firmware sees it (10 ms loop, five-sample speed window, PWM clamp, the firmware's PI law with conditional integration) and run the same identification algorithm on an open-loop excitation record. The network inputs are these **identified** values, not the true ones, so it learns to correct the bias of the on-device identifier.
3. Label each motor with the PI gains that minimise the cost below, found by a 36 × 36 logarithmic grid search followed by local refinement.
4. Train a 3-16-16-2 multi-layer perceptron (tanh hidden layers, inputs log K, log τ, d; outputs log Kp, log Ki) with Adam [28] on 3,200 motors, keeping 800 for validation, and export the weights as a C header.

The cost for a step sequence of 45 %, 80 % and 30 % of the motor's maximum speed is the time-weighted absolute error plus an overshoot penalty that becomes steep beyond 5 % and a small control-activity term:

$$J = \Sigma\,(0.2 + t)\cdot|e|/y_{max}\cdot T_s + 10\cdot OS^2 + 5\cdot\max(OS - 0.05,\,0) + 0.002\cdot\Sigma|\Delta u|/255 \qquad (3.6)$$

A two-hidden-layer network is a universal approximator for smooth mappings of this kind [29]; with 370 weights (about 1.5 kB) its forward pass takes a few hundred multiply–accumulate operations, negligible at 100 Hz. On the ESP32 both the lambda and neural gains are recomputed from every new estimate, clamped to safe ranges and smoothed with a time constant of about one second so the gains never jump.

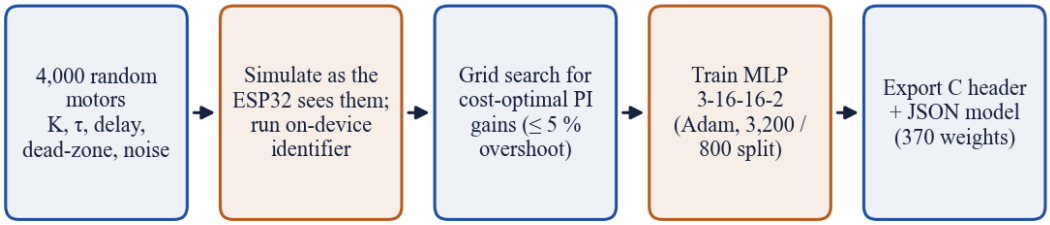


Figure 3.4: Offline training pipeline of the neural PI tuner

3.1.6 Anomaly detection with a digital twin

Once the identifier is confident, a copy of the learned model runs in parallel with the motor and is fed the same PWM, forming a minimal digital twin [24]:

$$\hat{y}[k] = a\cdot\hat{y}[k-1] + b\cdot u[k-d] + c \qquad (3.7)$$

During the first six seconds of driven operation the detector learns a **healthy baseline**: the mean and spread of the twin residual $r = y - \hat{y}$; a ridge-regularised linear fit of motor current against speed and PWM; and the mean and spread of vibration in each 40 RPM speed band (Welford's algorithm). The twin model is then frozen. Thereafter every sample is scored with z-scores against this baseline, smoothed over about 0.3 s:

$$z_s = (r - \mu_r)/\sigma_r, \quad z_i = (i - \hat{i}(y,u))/\sigma_i, \quad z_v = (v - \mu_{v,band})/\sigma_{v,band} \qquad (3.8)$$

Two further indicators complete the picture: the drift of the live identified K from its baseline value, and the thermal margin to the over-temperature limit. The class is decided from the pattern of signs (Figure 3.5): a motor slower than its twin while drawing more current is mechanically loaded; slower without extra current indicates a drive or supply loss; faster than the twin indicates a released load. A new class must persist for 0.3 s (vibration 1 s, return to

normal 0.5 s) before it is reported, and a continuous severity score in [0, 1] is reported alongside. No fault examples are needed at any stage, in contrast with supervised edge classifiers [26].

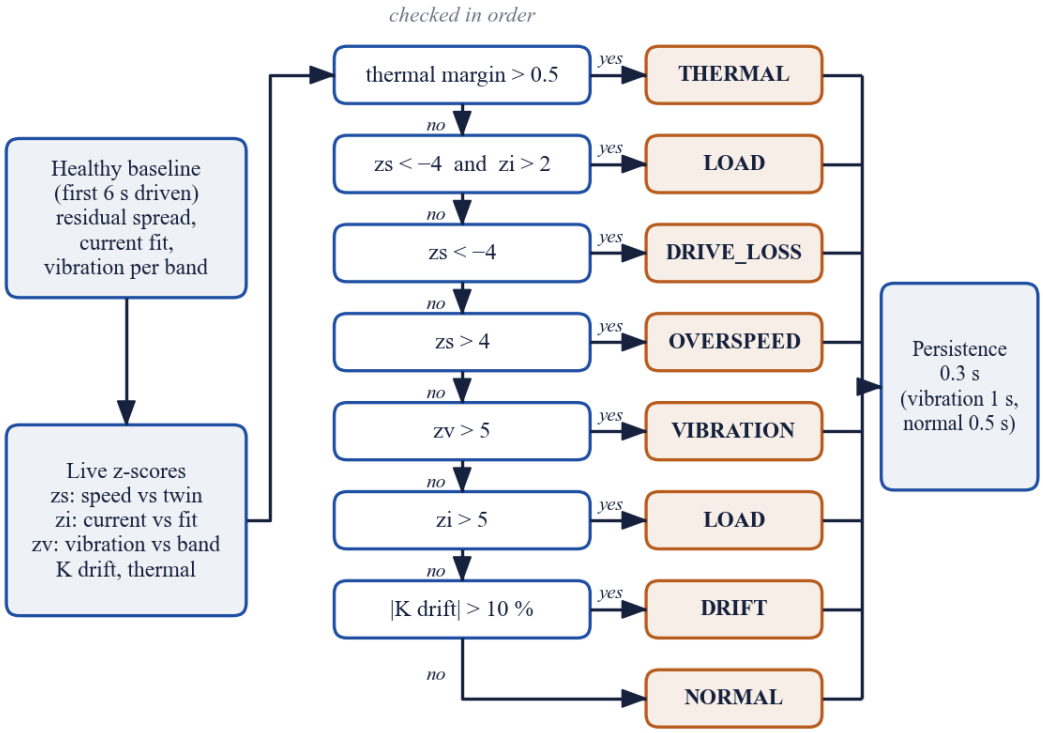


Figure 3.5: Anomaly classification from z-scores, drift and thermal margin

3.1.7 Safety layer

The anomaly detector is advisory. Hard limits in the control task cut the motor independently and latch until cleared by the operator:

Table 3.1: Hard protection limits in the firmware

Condition	Trip rule	Action
Over-current (instant)	$i \geq 2.0$ A	Disable bridge, latch OVERCURRENT
Over-current (sustained)	$i > 1.2$ A for 200 ms (TB6612FNG rating)	Disable bridge, latch OVERCURRENT
Stall	PWM > 150 and speed < 15 RPM for 0.5 s	Disable bridge, latch STALL
Over-temperature	Winding temperature > 58 °C	Disable bridge, latch OVERHEAT
Host-link loss	No line from host for 1.5 s	Disarm motor (heartbeat every 0.5 s)
Emergency stop	Dashboard E-stop or STOP MOTOR	Disable bridge, reset integrator

3.2 Technical Descriptions

3.2.1 Hardware setup

Figure 3.6 shows the experimental rig. The motor is a JGA12-N20 12 V, 300 RPM gear motor with a magnetic quadrature encoder (7 pulses per motor revolution per channel, 1:100 gearbox), giving 2,800 counts per output revolution in full quadrature. A TB6612FNG MOSFET H-bridge [30] drives it from a 12 V supply with a 20 kHz, 10-bit PWM. An INA219 [31] in series with one motor lead measures current through a 0.1 Ω shunt, an MPU6050 [32] fixed to the side of the motor measures vibration, a DS18B20 probe is fixed to the motor can beside it, and an SSD1306 OLED shows the live state. All I²C devices share one 400 kHz bus. A 100 µF electrolytic capacitor across the driver supply and a 0.1 µF ceramic capacitor across the motor terminals suppress the switching and brush noise that initially caused USB disconnections.

The electrical connections are summarised in Figure 3.7 and Table 3.2.

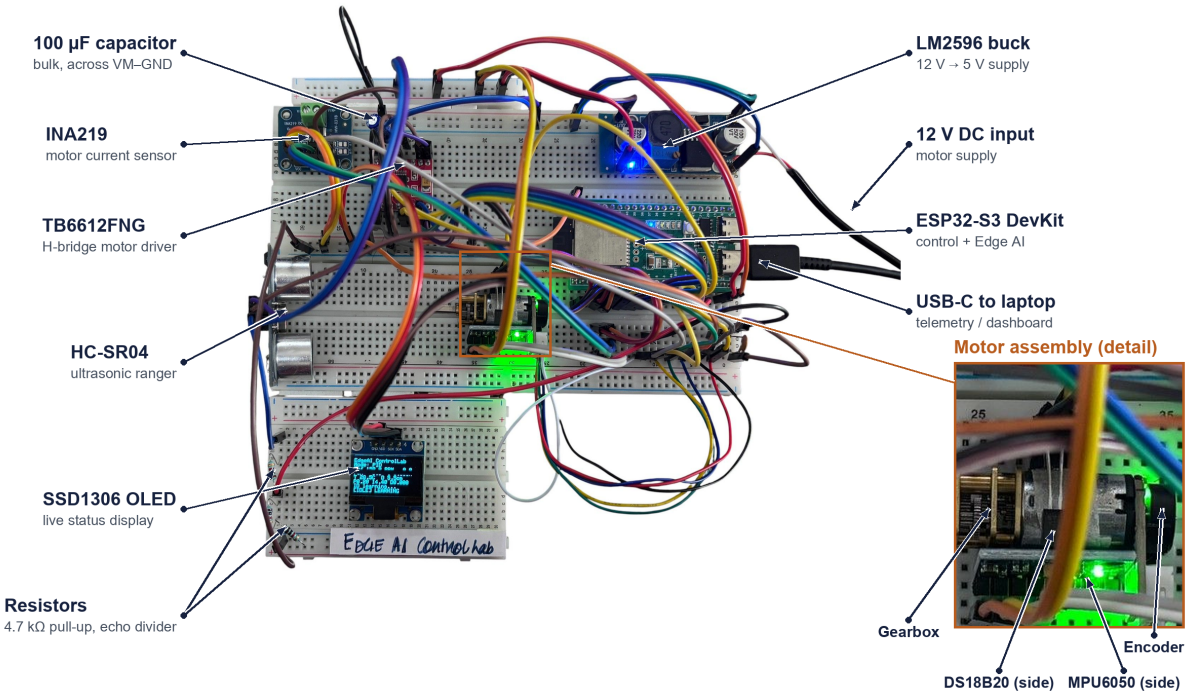


Figure 3.6: Photograph of the bench rig with the main components labelled; the inset shows the DS18B20 and MPU6050 mounted on the side of the motor

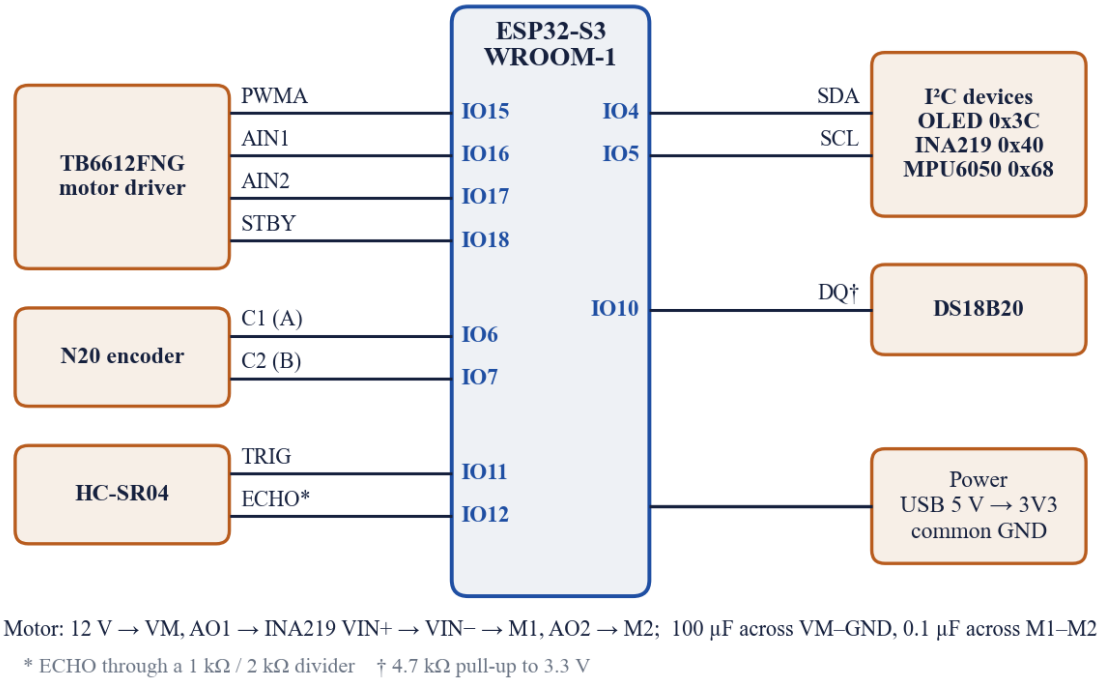


Figure 3.7: Wiring of the ESP32-S3 to the driver, sensors and motor

Table 3.2: ESP32-S3 pin allocation

Signal	GPIO	Connected to	Notes
I²C SDA / SCL	IO4 / IO5	OLED (0x3C), INA219 (0x40), MPU6050 (0x68)	400 kHz shared bus
Encoder A / B	IO6 / IO7	Motor C1 / C2	PCNT hardware counter, ×4 decoding
1-Wire	IO10	DS18B20 DQ	4.7 kΩ pull-up to 3.3 V
TRIG / ECHO	IO11 / IO12	HC-SR04	ECHO through 1 kΩ / 2 kΩ divider
PWMA	IO15	TB6612FNG PWMA	LEDC, 20 kHz, 10-bit
AIN1 / AIN2	IO16 / IO17	TB6612FNG direction	Forward = 1 / 0, coast = 0 / 0
STBY	IO18	TB6612FNG standby	High = enabled; low on fault

3.2.2 Firmware architecture

The firmware is written in C++ with the Arduino core on FreeRTOS [33] and built with PlatformIO. It uses both cores of the ESP32-S3 [34]. A high-priority **control task** pinned to core 1 runs every 10 ms with vTaskDelayUntil; its sequence is shown in Figure 3.8. The Arduino **loop** on core 1 parses commands and streams telemetry from a queue, so a slow host can never delay the control loop (samples are dropped instead). A low-priority **aux task** on core 0 refreshes the OLED, polls the 1-Wire sensor and fires the ultrasonic ranger, and every second checks that the OLED still acknowledges and re-initialises it after a supply glitch.

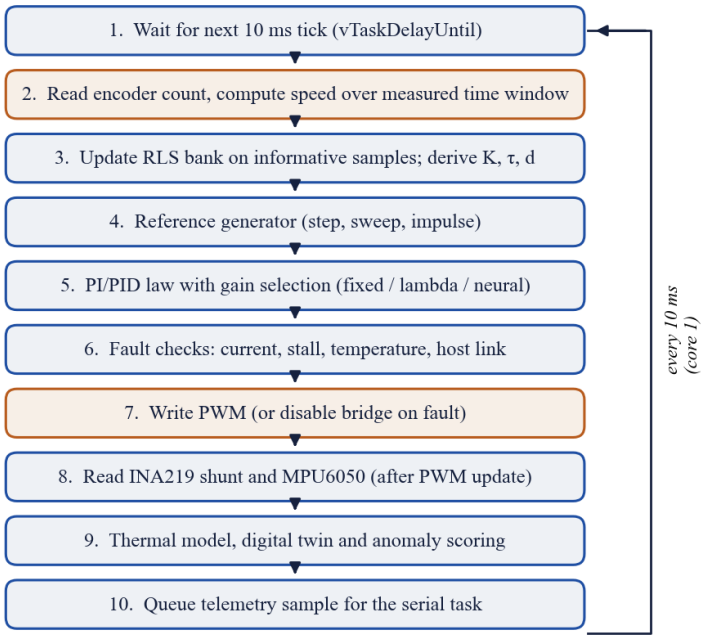


Figure 3.8: Sequence executed by the 100 Hz control task in every period

Slow I²C reads are deliberately placed after the PWM update, so a bus wait affects only the age of the current sample, not the timing of the actuation. The INA219 is read with a single shunt-register transaction instead of the library's three. The firmware occupies 20.7 kB of static RAM (6.3 %) and 346 kB of flash (10.4 %).

3.2.3 Communication protocol

The ESP32-S3 enumerates as a native USB CDC serial port. Every control sample is sent as one JSON line of about 440 bytes; the main fields are listed in Table 3.3. The host sends the dashboard's control packet ({"kp", "ki", "kd", "targetSpeed", "controllerMode", "excitationSignal", "isEStop", "tuner"}) or text commands such as status, scan, idreset, baseline, tuner nn|lambda and ping. A status line prefixed with "# info" carries a machine-readable description of the firmware build, sensors and limits for the dashboard.

Table 3.3: Main telemetry fields (one JSON object per 10 ms sample)

Field(s)	Meaning
time, ref, output, input, error	Time, setpoint (RPM), measured speed (RPM), PWM, tracking error
current, volt, vib, temp, dist	Motor current (A), supply (V), vibration (g RMS), winding temperature (°C), distance (cm)
fault, mode, gKp / gKi / gKd	Protection state, controller mode, gains applied in this period
idK, idTau, idD, idN, idOk	Identified gain, time constant, delay, number of updates, confidence flag
aKp, aKi, nKp, nKi, tun	Lambda-rule gains, neural-tuner gains, active tuner
an, acl, zs, zi, zv, kdr	Anomaly score and class, z-scores (speed, current, vibration), K drift (%)
twin, abp, therm	Digital-twin speed, baseline progress, thermal margin

3.2.4 Backend server

The backend (Python, FastAPI) opens the serial port with the control lines set so that opening it does not reset the board into its ROM bootloader, forwards telemetry to browsers over a WebSocket, forwards control packets to the ESP32, and sends a heartbeat every 0.5 s that keeps the firmware watchdog satisfied. A link watchdog re-enables telemetry and resets the board if the stream stops, and a status poller refreshes the hardware description every 5 s. REST endpoints provide batch identification of logged data (first- and second-order models with numpy), the same neural tuner for what-if analysis, the model card, and a SQLite store of saved runs. When no serial port is configured the backend runs a **simulator** whose plant (K = 1.2 RPM/PWM, τ = 60 ms, three-sample delay), current and thermal behaviour and control law match the firmware, so the whole dashboard can be used without hardware.

3.2.5 Laboratory dashboard and 3D digital twin

The dashboard (React, TypeScript, Vite, three.js) has ten tabs, all driven by live data. The overview (Figure 3.9) shows an oscilloscope of setpoint, speed and PWM, runtime metrics and the current plant model. Display updates are batched at 25 Hz while the full 100 Hz stream is recorded into a 60 s buffer, which reduced the measured round-trip latency from 47 ms to under 2 ms on the development laptop.

The **Digital Twin 3D** tab (Figure 3.10) renders the N20 motor, gearbox, encoder magnet and Hall sensors, wiring and the MPU6050 bracket. The shaft and encoder spin at the measured speed, the assembly shakes with the measured vibration, the can glows with temperature, current flows visibly in the leads and an anomaly halo appears when the detector raises a class.

A gauge compares the measured speed, the setpoint and the twin's prediction.

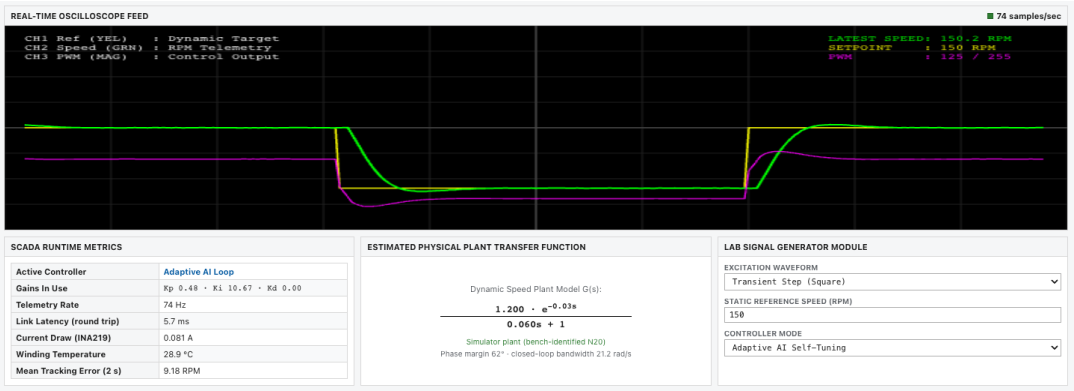


Figure 3.9: Dashboard overview: setpoint, speed and PWM during step excitation in Adaptive AI mode

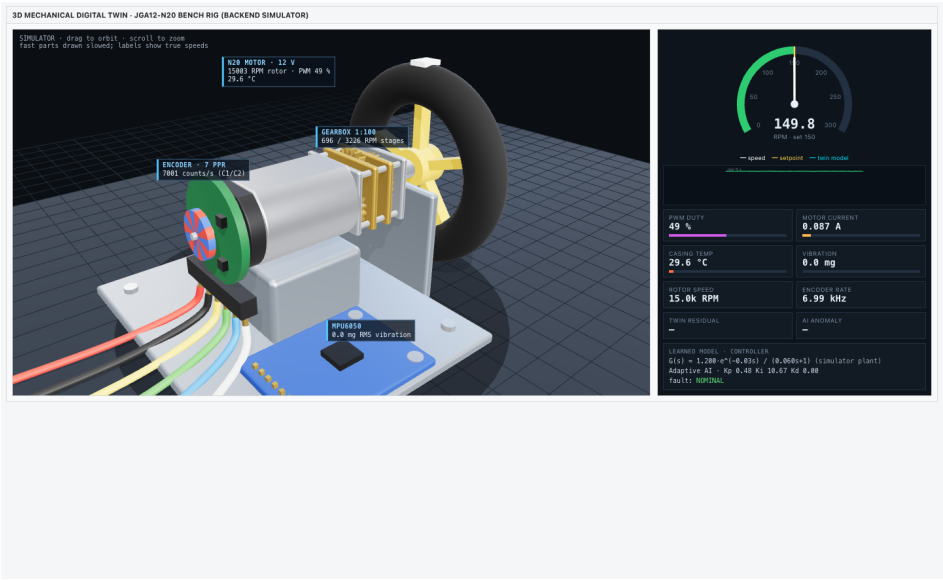


Figure 3.10: Digital Twin 3D tab: rendered rig driven by telemetry, with speed gauge and live readouts

The **Controller Tuning** tab sets gains, controller mode and the adaptive gain source (neural tuner or lambda rule) and shows the control law currently applied on the ESP32. The **AI Predictor** tab shows the live neural and lambda gains with the network's model card and permutation feature importance; **System Identification** fits first- and second-order models to recorded data and plots their poles; **Fault Alarms** shows protection limits, anomaly z-scores and an alarm timeline; **Frequency Response** and **State Space** analyse the identified loop; **Saved Runs** and **Export Reports** store runs in SQLite and generate CSV, LaTeX and printable PDF reports. The AI Predictor tab is shown in Figure 3.11.

3.2.6 Virtual sensors

Two of the planned sensors were not operational at the time of writing: the DS18B20 probe returned no presence pulse and the HC-SR04 returned no echo. So that the protection logic

and the dashboard remain complete, the firmware replaces them with clearly flagged substitutes (telemetry fields tsrc = "model" and dsrc = "fixed"). Temperature comes from a first-order **thermal model** driven by measured quantities: copper loss i²R (R = 40 Ω) plus a speed-proportional friction loss of 0.1 W at 300 RPM, through a thermal resistance of 25 K/W, heating with a 90 s time constant and cooling more slowly with 240 s, from an ambient of 28.5 °C (Figure 3.12). Distance is held at a fixed 5.5 cm. The firmware switches to the real DS18B20 automatically as soon as one is detected on the bus, which it re-probes every two seconds.

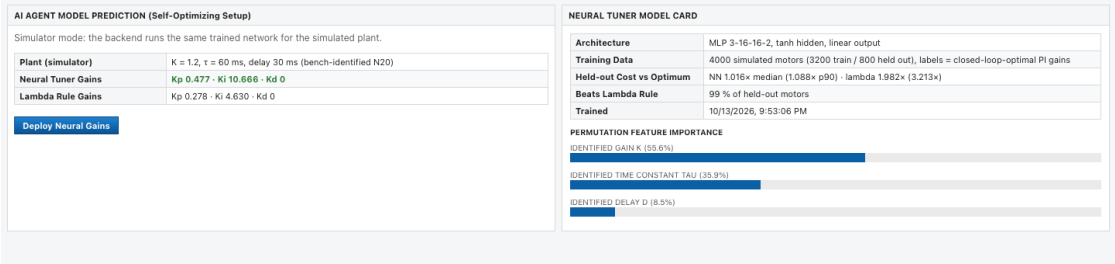


Figure 3.11: AI Predictor tab: neural-tuner gains, lambda-rule gains and model card with feature importance

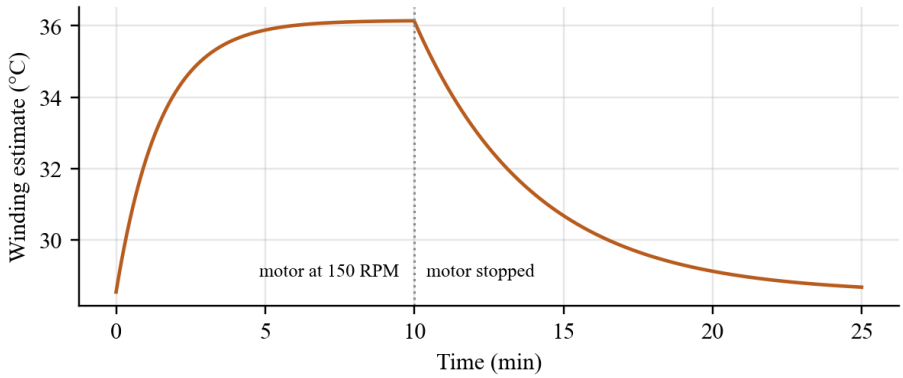


Figure 3.12: Thermal model used as a virtual temperature sensor: 10 min at 150 RPM followed by cooling

4. HARDWARE/SOFTWARE TOOLS USED

The hardware was chosen for low cost, availability and 3.3 V logic compatibility; the software stack is entirely open source.

Table 4.1: Hardware components

Component	Specification	Role
ESP32-S3 DevKit (WROOM-1)	Dual-core Xtensa LX7, 240 MHz, 512 kB SRAM, native USB	Control, identification, tuning, anomaly detection
JGA12-N20 gear motor	12 V, 300 RPM, 1:100, magnetic encoder 7 PPR	Plant under control
TB6612FNG driver	Dual MOSFET H-bridge, 1.2 A continuous, 3.2 A peak	PWM motor drive
INA219 module	12-bit, I²C, 0.1 Ω shunt, ±3.2 A	Motor current and supply voltage
MPU6050 module	3-axis accelerometer ±4 g, I²C	Vibration measurement
SSD1306 OLED	0.96 in, 128 × 64, I²C	Local status display
DS18B20	1-Wire, −55 to 125 °C, ±0.5 °C	Winding temperature (planned)
HC-SR04	2–400 cm ultrasonic ranger	Distance channel (planned)
LM2596 buck converter	4.5–40 V in, adjustable out (5 V)	Logic supply from 12 V
12 V 2 A adapter, breadboard, capacitors	100 µF bulk, 0.1 µF motor snubber	Power and noise suppression

Table 4.2: Software tools

Tool	Version / detail	Used for
PlatformIO + Arduino-ESP32 core	espressif32 6.5.0, core 2.0.14	Firmware build and flashing
FreeRTOS	Bundled with ESP-IDF	Tasks, queues, timing
ArduinoJson, ESP32Encoder	7.x, 0.11.7	Telemetry and commands; PCNT encoder decoding
Adafruit INA219 / SSD1306 / GFX	Latest stable	Sensor and display drivers
Python, NumPy	3.x, 2.2.6	Neural-tuner training, batch identification
FastAPI, Uvicorn, pySerial	pySerial 3.5	Backend, WebSocket, serial bridge
SQLite	Built into Python	Saved runs
React, TypeScript, Vite	React 18, Vite dev server	Dashboard
three.js	OrbitControls, CSS2DRenderer	3D digital twin
Matplotlib	3.10	Report figures from logged data

5. RESULT ANALYSIS

Results were obtained on the bench motor at 12 V unless stated otherwise. Settling time uses a 2 % band around the new setpoint; overshoot is relative to the step size.

5.1 Hardware Bring-up and Open-loop Characterisation

All three I²C devices enumerated (0x3C, 0x40, 0x68). In raw-PWM mode the motor reached 304 RPM at full duty with no load, consistent with a gain of about 1.2 RPM per PWM step, and drew 40–260 mA in normal closed-loop operation (mean 44 mA over a 43 s stress run). Bring-up exposed several practical issues whose fixes are now part of the design: USB disconnections caused by motor noise (cured by a star ground and the 100 µF and 0.1 µF capacitors, and made safe by the host-link watchdog); spurious stall faults while idle (faults are now judged only while driving); speed spikes caused by I²C contention (timestamped speed estimation); and a false 1.64 A current reading when the INA219 was moved to the PWM-switched high side of the bridge (it is kept on the low-side leg).

5.2 System Identification Results

Figure 5.1 shows a logged hardware record in which the setpoint alternates between 50 and 150 RPM every second. The same informative-sample gating as on the device was applied to this record and the model $y[k] = a \cdot y[k-1] + b \cdot u[k-d] + c$ was fitted for each candidate delay. Table 5.1 and Figure 5.2 compare the delays.

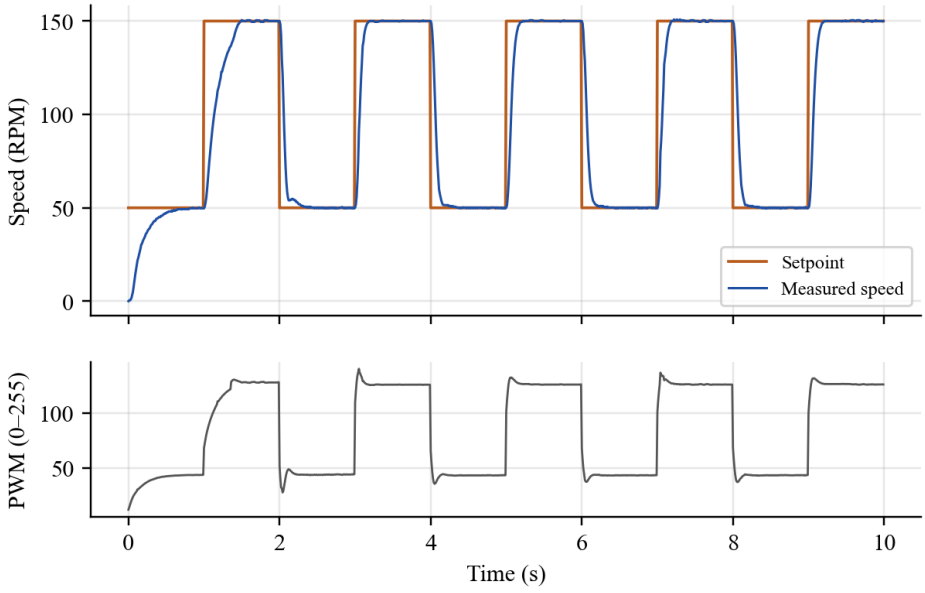


Figure 5.1: Logged hardware response to 50 ↔ 150 RPM steps in Adaptive AI mode: speed (top) and PWM (bottom)

Table 5.1: Identification of the bench motor for each candidate delay (264 informative samples)

Delay d (samples)	K (RPM/ PWM)	τ (ms)	One-step RMS error (RPM)	Free-run fit (%)
1	1.433	93.6	2.35	83.1
2	1.308	72.5	1.79	91.8
3	**1.208**	**56.6**	**1.53**	**96.6**
4	1.130	45.4	2.01	92.6
5	1.075	38.0	2.88	87.8
6	1.031	39.0	4.09	82.1

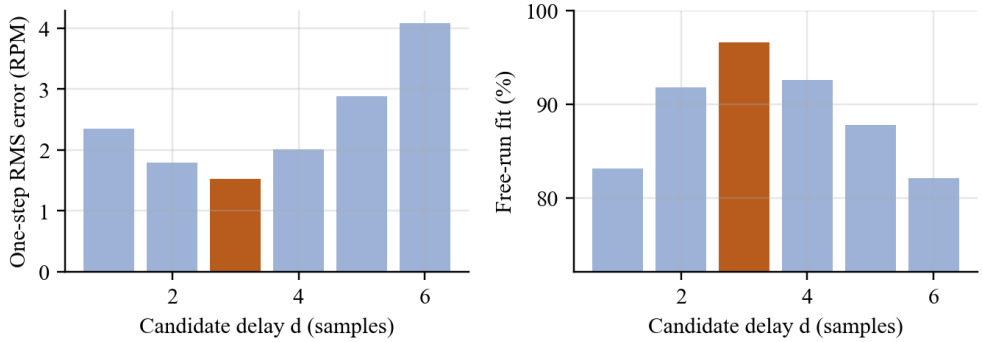


Figure 5.2: Delay bank on bench data: one-step prediction error (left) and free-run fit (right); d = 3 is best on both

Both criteria select d = 3 samples (30 ms). The resulting model reproduces the measured speed with a 96.6 % free-run fit (Figure 5.3), whereas the conventional assumption d = 1 overestimates K by 19 % and τ by 65 % and fits only 83.1 %. Over repeated on-device runs the estimates were K ≈ 1.18–1.24 RPM/PWM, τ ≈ 60–72 ms and d = 3, in agreement with this batch analysis; the confidence gate typically opened within one to two seconds of step excitation. The identified gain also matches the open-loop observation of 304 RPM at PWM 255.

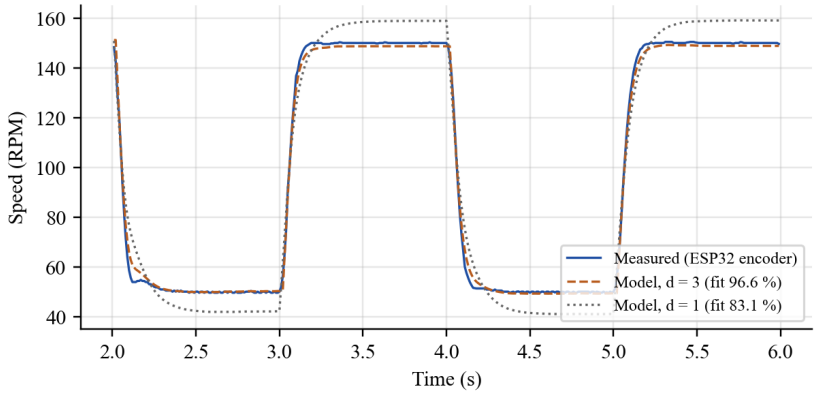


Figure 5.3: Measured speed versus free-run simulation of the identified models with d = 3 and d = 1

5.3 Neural Tuner: Offline Evaluation

Of the 4,000 training motors the on-device identifier assigned delays between 2 and 6 samples (Figure 5.4, left); because of the speed window, identified delays are systematically larger than the true ones, which is exactly why the network is trained on identified rather than true parameters. Training took 97 s on a laptop and reached a normalised validation error of 0.025. An earlier version without the delay input reached only 0.144 and overshot by up to 18 % on long-delay plants. Permutation importance (right) shows that the gain K matters most (55.6 %), then τ (35.9 %), then d (8.5 %).

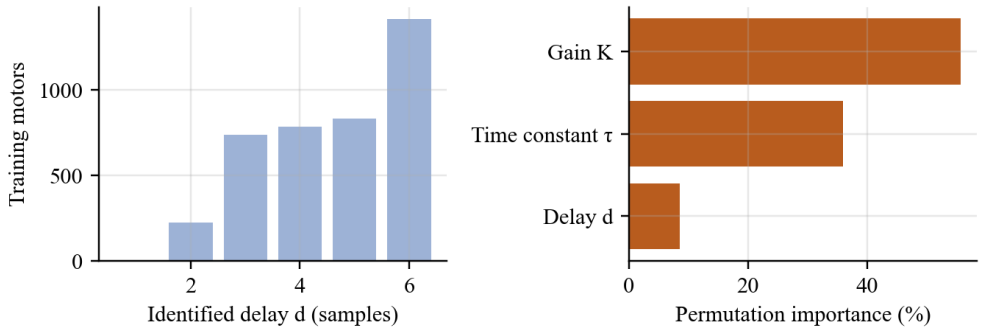


Figure 5.4: Training data: identified-delay distribution (left) and permutation feature importance (right)

To test generalisation, 300 further motors that were never used in training or validation were generated and evaluated in closed loop (Figure 5.5 and Table 5.2). The network's gains came within a median of 1.6 % of the grid-search optimum and were better than the lambda rule on 99.3 % of motors; the lambda rule's median cost was twice the optimum.

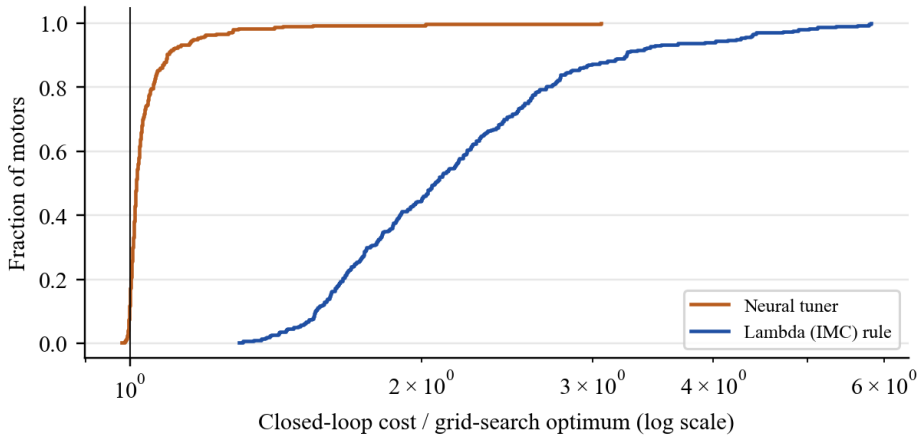


Figure 5.5: Closed-loop cost relative to the optimum on 300 unseen simulated motors

Table 5.2: Generalisation of the tuners (cost relative to grid-search optimum)

Measure	Lambda rule	Neural tuner
Median cost ratio, 800 validation motors	1.982	1.016
90th percentile, 800 validation motors	3.213	1.088
Median cost ratio, 300 unseen motors	2.064	1.016
90th percentile, 300 unseen motors	3.253	1.093
Motors where the tuner is better	0.7 %	99.3 %
Motors within 5 % of optimum	—	78.7 %

Figure 5.6 shows the learned gain surfaces for d = 3. Both gains fall with increasing K, as expected, but the network's dependence on τ is not the proportional one of the lambda rule: it adds proportional gain for slow motors while holding the integral gain nearly constant across τ, something no single closed-form rule captures. For the bench motor (marked) it predicts Kp = 0.63 and Ki = 10.8, compared with 0.35 and 4.7 from the lambda rule.

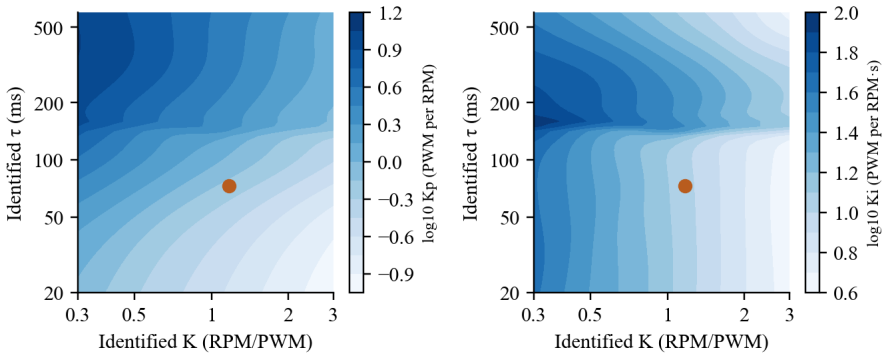


Figure 5.6: Neural-tuner gain maps for d = 3 (dot: bench motor, K = 1.17, τ = 73 ms)

5.4 Closed-loop Performance

The simulator was first validated against hardware: the hand-tuned gains (Kp = 0.2, Ki = 4.4) settle in 0.60 s in simulation and 0.55 s on the motor. A simulated motor whose identified model matches the bench motor (identified K = 1.17, τ = 72 ms, d = 3) was then driven through the training step sequence with each gain set (Figure 5.7). The neural tuner settled in 0.16–0.22 s with at most 2.1 % overshoot, against 0.65–0.70 s for the lambda rule and 0.58 s for the hand-tuned gains, and reduced the closed-loop cost by 61 %.

On the real motor the result was the same. In the logged run of Figure 5.1 the first step, taken with the default gains before the identifier was confident, settled in 0.45 s; once the model was accepted and the neural gains took over, every subsequent 50 ↔ 150 RPM step settled in 0.16–0.18 s with at most 0.9 % overshoot. Dedicated comparisons with the tuner switched live

between the two gain sources gave the values in Table 5.3.

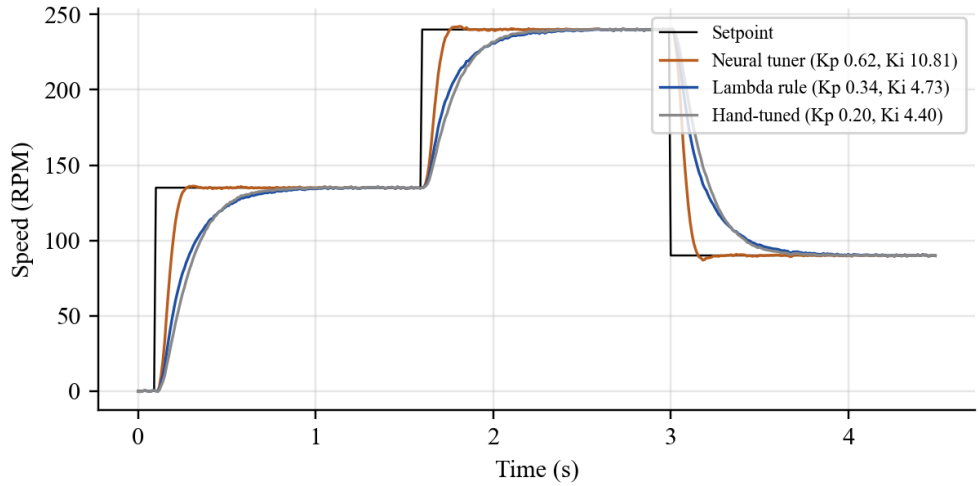


Figure 5.7: Simulated step sequence on a motor matched to the bench identification

Table 5.3: Real-motor comparison of the two adaptive gain sources

Test (bench motor, 12 V)	Lambda rule	Neural tuner
Step 120 → 230 RPM, settling (2 %)	0.53 s	0.15 s
Step 230 → 90 RPM, settling (2 %)	0.73 s	0.25 s
Step 50 ↔ 150 RPM, settling (logged run)	—	0.16–0.18 s
Maximum overshoot	≤ 0.6 %	≤ 1.3 %
Gains for K = 1.17, τ = 73 ms, d = 3	Kp = 0.35, Ki = 4.75	Kp = 0.63, Ki = 10.8

The neural tuner is therefore about 3.5 times faster than the analytic rule on hardware while keeping overshoot far below the 5 % requirement, confirming that the mapping learned from simulated motors transfers to a real one. The bumpless transfer worked as intended: switching gain sources mid-run produced no visible bump in speed.

5.5 Frequency-domain Verification

The dashboard's Frequency Response tab (Figure 5.8) evaluates the identified plant and the open loop with the gains in use. With the neural-tuner gains the loop has a phase margin of 62° and a gain margin of 13.6 dB, comfortably inside the usual design guidelines (PM ≥ 45°, GM ≥ 6 dB) [2]. The aggressive gains are thus fast without being fragile; the margins also explain the small overshoot observed.

The State Space tab gives the same loop in state-space form (state x = speed), confirms full controllability and observability, and lists the closed-loop poles with the PI integrator state added.

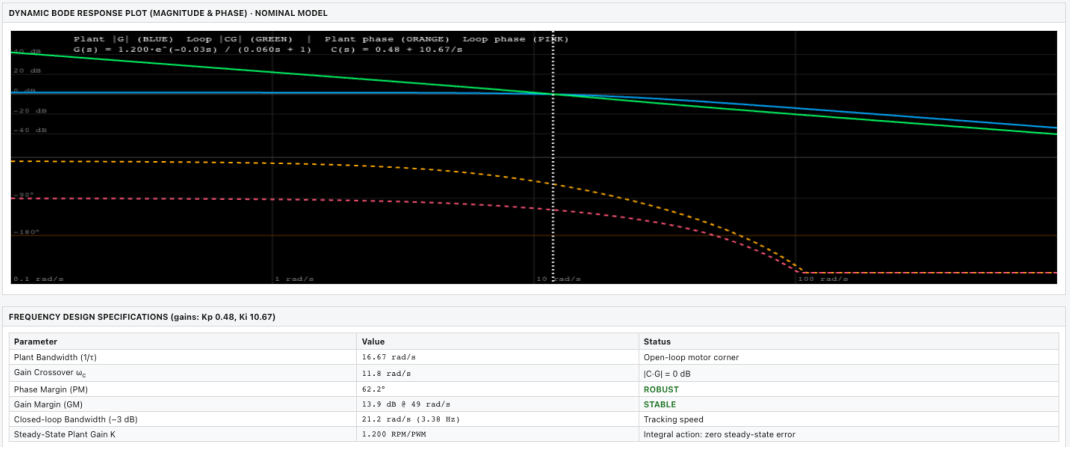


Figure 5.8: Bode plot of the identified plant and open loop, with phase and gain margins

5.6 Anomaly Detection

The first hardware stress test immediately revealed a property of the motor that a simple model misses: vibration is not monotonic in speed (Figure 5.9). It rises from about 7 mg RMS at 60 RPM to 26–28 mg between 180 and 220 RPM and falls again to about 20 mg at 260 RPM, and it is bursty. The original baseline modelled vibration as a linear function of speed and PWM.

In that first 43 s stress run (Figure 5.10) the linear baseline produced a 0.6 s false VIBRATION alarm when the motor first entered the 180–200 RPM resonance region at t ≈ 25 s. The baseline was therefore made per speed band, bands not seen during the baseline now learn from their first 1.5 s instead of being extrapolated into, and vibration requires 1 s of persistence. After these changes, two 30 s stress runs covering steps, sine sweeps, 80–260 RPM jumps and controller-mode switches produced **no false diagnoses**, with all z-scores remaining inside their thresholds.

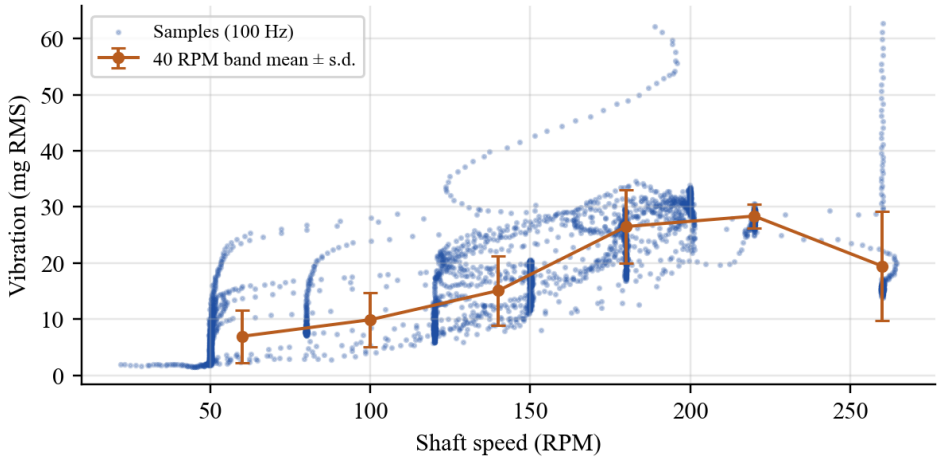


Figure 5.9: Measured vibration versus speed on the bench motor, with 40 RPM band means

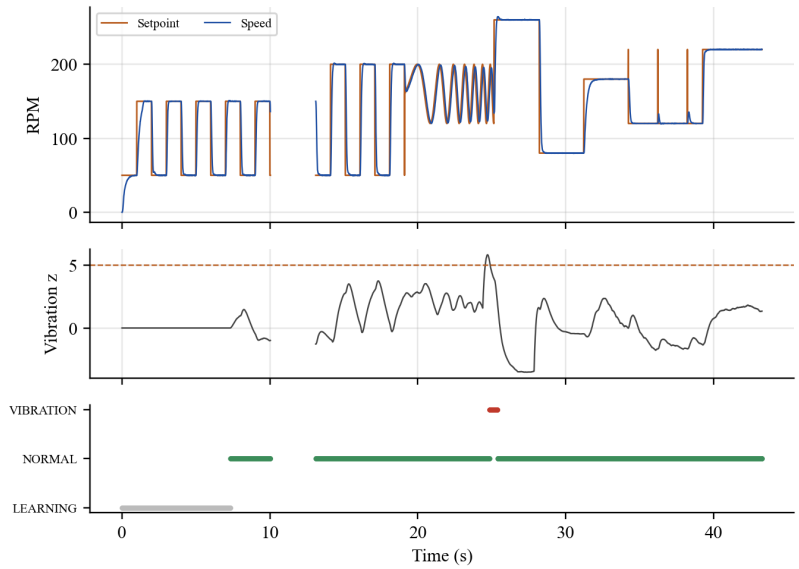


Figure 5.10: First hardware stress run with the linear vibration baseline: speed, vibration z-score and reported class

The fault logic itself was exercised in simulation and through the live z-score display: a slowed motor drawing more current moves zs negative and zi positive, which the classifier reports as LOAD after 0.3 s. A deliberate fault-injection test on the bench (braking the shaft, lowering the supply) has not yet been carried out and is listed as future work; the false-alarm result therefore establishes specificity but not yet sensitivity on hardware.

5.7 Computational Footprint

Table 5.4: Resource use on the ESP32-S3

Item	Value
Static RAM / flash	20.7 kB (6.3 %) / 346 kB (10.4 %)
Control rate	100 Hz on core 1 (identification, tuning and anomaly detection included)
RLS bank	6 estimators × 3 parameters, 3 × 3 covariance each
Neural tuner	MLP 3-16-16-2, 370 weights (≈ 1.5 kB), ≈ 370 MACs + 32 tanh per evaluation
Anomaly detector	3 running statistics, 3 × 3 least-squares fit, 8 vibration bands
Telemetry	≈ 440 bytes per sample, 100 samples/s over native USB
Dashboard latency	< 2 ms round trip after display batching (47 ms before)

All three learning functions together fit in a few kilobytes and run every 10 ms without disturbing the loop, confirming that model identification, gain prediction and anomaly detection are well within the capacity of a microcontroller costing a few hundred rupees.

5.8 Discussion

Three findings stand out. First, **dead time dominates**: identifying the delay raised the model fit from 83 % to 97 % and halved the validation error of the tuner; any learned tuner that ignores it will be either slow or oscillatory. Second, **learning the mapping, not the plant**, is what makes the approach practical: the network is trained once, offline, in about a minute and a half, yet it tunes a motor it has never seen within seconds of excitation, in contrast with RL tuners that must interact with each plant [17], [18]. Third, **unsupervised monitoring needs physically informed baselines**: the per-speed-band vibration model was the difference between a nuisance alarm and a quiet detector.

6. CONCLUSION AND FUTURE WORK

6.1 Summary

EdgeAI ControlLab demonstrates that a low-cost microcontroller can identify, tune and monitor a DC motor on its own. A bank of RLS estimators on the ESP32-S3 learned the bench motor's gain, time constant and three-sample dead time with a 96.6 % fit; a 370-weight neural network trained on simulated motors converted that model into PI gains that settle speed steps in 0.15–0.27 s, about 3.5 times faster than the lambda rule, with at most 1.3 % overshoot and 62° of phase margin; and a digital-twin anomaly detector learned a healthy baseline in six seconds and ran through stress tests without false alarms. The system uses about 6 % of the controller's RAM, costs about ₹3,100 and comes with a laboratory dashboard and 3D digital twin.

6.2 Limitations and Constraints

- The DS18B20 and HC-SR04 were not operational; temperature comes from a thermal model and distance is a fixed placeholder, so thermal protection is model-based until the probe is fitted.
- Detection of real faults has not yet been demonstrated on hardware; only the absence of false alarms has.
- The first-order-plus-delay model only approximates strongly second-order or nonlinear loads, and only one motor type was tested on hardware (other motors in simulation).
- Adaptive mode is PI only and unidirectional: D action biased identification, and the INA219 must stay on the low-side leg.

6.3 Improvement / Future Work

- Fault-injection experiments (shaft braking, added inertia, supply sag, loose mounting) to measure detection sensitivity and delay for each anomaly class.
- Repair the DS18B20 and HC-SR04 connections and validate the thermal model against measured temperature.
- Extend the tuner to second-order models and PID gains, quantise it to int8 [27] and test other motors and plants.

7. SOCIAL AND ENVIRONMENTAL IMPACT

Education. Control engineering is often taught with simulations because commercial laboratory rigs are costly. A complete identification, adaptive-control and condition-monitoring rig for about ₹3,100, with an open dashboard and a simulator mode, allows every student or small group to work with real hardware and to see the internal quantities of an adaptive controller as it learns.

Energy. Motor-driven systems use a large fraction of the world's electricity [1]. A well-tuned loop reaches its setpoint quickly without oscillation, which avoids wasted control effort and heating, and an anomaly detector that notices rising current for the same speed points to friction, misalignment or wear that waste energy long before they cause a breakdown.

Industry and maintenance. Small and medium manufacturers rarely have control specialists to retune every drive. Self-tuning drives that also report their own health reduce commissioning time and enable predictive rather than reactive maintenance [23], reducing downtime, spare-part consumption and electronic waste from prematurely replaced equipment.

Privacy and resilience. Because learning happens on the device, no operating data need leave the machine and the controller keeps working when the network does not. The low power consumption of the microcontroller (well under a watt) compared with a PC or cloud service further reduces the environmental footprint of adding intelligence to a drive.

Safety. Hard protection limits operate independently of the learning layer, and the motor stops automatically when the supervising computer is disconnected, so adding AI does not weaken the safety of the system.

8. WORK PLAN

8.1 Timeline

The project was carried out between July and November 2026 (Figure 8.1). Software development began with the dashboard and simulator so that the user interface was ready when the hardware arrived; the Edge-AI functions were developed once the control loop had been validated on the bench.

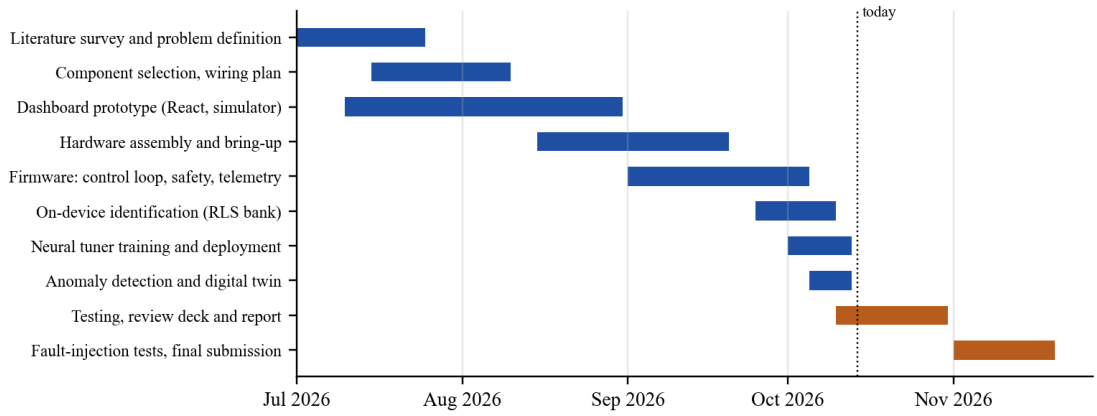


Figure 8.1: Project timeline (blue: completed; orange: remaining)

8.2 Individual Contribution

Table 8.1: Individual contributions

Team member	Contribution
Pratik Kumar (24BML0027)	System architecture; ESP32-S3 firmware (control loop, safety, telemetry); on-device RLS identification; neural-tuner training pipeline and deployment; anomaly detector; hardware bring-up and bench experiments
Thakur Chand Choudhary (24BML0012)	Hardware assembly and wiring; sensor integration and noise suppression; FastAPI backend and simulator; React dashboard and 3D digital twin; test campaigns, documentation, review presentation and report

9. COST ANALYSIS

Table 9.1 lists the components with approximate retail prices from Indian online electronics stores (2026). Prices vary between sellers by roughly ±20 %.

Table 9.1: Bill of materials and cost

Component	Qty	Unit price (₹)	Amount (₹)
ESP32-S3 DevKit (WROOM-1)	1	850	850
JGA12-N20 12 V 300 RPM encoder gear motor	1	550	550
TB6612FNG motor-driver module	1	180	180
INA219 current-sensor module	1	180	180
MPU6050 accelerometer/gyroscope module	1	160	160
SSD1306 0.96 in OLED display	1	220	220
DS18B20 temperature probe	1	120	120
HC-SR04 ultrasonic sensor	1	80	80
LM2596 buck converter	1	100	100
12 V 2 A DC adapter	1	250	250
Breadboards and jumper wires	1 set	250	250
Capacitors, resistors, motor bracket, fasteners	1 set	150	150
Total			**3,090**

The software is entirely open source and the dashboard runs on any laptop, so there is no licence cost. For comparison, commercial educational servo-control trainers typically cost several thousand US dollars per station. The ESP32-S3 provides the identification, tuning and monitoring functions at no additional hardware cost; a conventional approach would need a PC with control software in the loop for the same functions.

10. REFERENCES

[1] P. Waide and C. U. Brunner, “Energy-efficiency policy opportunities for electric motor-driven systems,” International Energy Agency, Paris, France, IEA Energy Papers no. 2011/07, 2011.

[2] K. J. Åström and T. Hägglund, *Advanced PID Control*. Research Triangle Park, NC, USA: ISA, 2006.

[3] J. G. Ziegler and N. B. Nichols, “Optimum settings for automatic controllers,” *Trans. ASME*, vol. 64, pp. 759–768, 1942.

[4] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA, USA: O’Reilly Media, 2019.

[5] P. P. Ray, “A review on TinyML: State-of-the-art and prospects,” *J. King Saud Univ.—Comput. Inf. Sci.*, vol. 34, no. 4, pp. 1595–1623, 2022.

[6] D. E. Rivera, M. Morari, and S. Skogestad, “Internal model control: PID controller design,” *Ind. Eng. Chem. Process Des. Dev.*, vol. 25, no. 1, pp. 252–265, 1986.

[7] S. Skogestad, “Simple analytic rules for model reduction and PID controller tuning,” *J. Process Control*, vol. 13, no. 4, pp. 291–309, 2003.

[8] A. O’Dwyer, *Handbook of PI and PID Controller Tuning Rules*, 3rd ed. London, U.K.: Imperial College Press, 2009.

[9] L. Ljung, *System Identification: Theory for the User*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1999.

[10] L. Ljung and T. Söderström, *Theory and Practice of Recursive Identification*. Cambridge, MA, USA: MIT Press, 1983.

[11] S. Haykin, *Adaptive Filter Theory*, 4th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2002.

[12] K. J. Åström and B. Wittenmark, *Adaptive Control*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1995.

[13] T. R. Fortescue, L. S. Kershenbaum, and B. E. Ydstie, “Implementation of self-tuning regulators with variable forgetting factors,” *Automatica*, vol. 17, no. 6, pp. 831–835, 1981.

[14] N. H. Sahrir and M. A. M. Basri, “Intelligent PID controller based on neural network for AI-driven control quadcopter UAV,” *Int. J. Robot. Control Syst.*, vol. 4, no. 2, pp. 691–708, 2024, doi: 10.31763/ijrcs.v4i2.1374.

[15] Z. Zhou, Y. Lu, S. Kokubu, P. E. Tortós, and W. Yu, “A GAN based PID controller for highly adaptive control of a pneumatic-artificial-muscle driven antagonistic joint,” *Complex Intell. Syst.*, vol. 10, pp. 6231–6248, 2024, doi: 10.1007/s40747-024-01488-y.

[16] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[17] G. Bujgoi and D. Sendrescu, “Tuning of PID controllers using reinforcement learning for nonlinear system control,” *Processes*, vol. 13, no. 3, Art. no. 735, 2025, doi: 10.3390/pr13030735.

[18] X. Hao *et al.*, “Deep reinforcement learning enhanced PID control for hydraulic servo systems in injection molding machines,” *Sci. Rep.*, vol. 15, Art. no. 23005, 2025, doi: 10.1038/s41598-025-05904-2.

[19] R. Shahouni, M. Bahraini, M. Abrofarakh, and M. Abbasi, “Adaptive tuning of fractional order PID controllers for nonlinear processes using hybrid PSO DQN reinforcement learning,” *Sci.*

Rep., vol. 15, Art. no. 38545, 2025, doi: 10.1038/s41598-025-22509-x.

[20] R. Isermann, “Model-based fault-detection and diagnosis—Status and applications,” *Annu. Rev. Control*, vol. 29, no. 1, pp. 71–85, 2005.

[21] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Comput. Surv.*, vol. 41, no. 3, Art. no. 15, Jul. 2009.

[22] Y. Lei, B. Yang, X. Jiang, F. Jia, N. Li, and A. K. Nandi, “Applications of machine learning to machine fault diagnosis: A review and roadmap,” *Mech. Syst. Signal Process.*, vol. 138, Art. no. 106587, 2020.

[23] R. K. Mobley, *An Introduction to Predictive Maintenance*, 2nd ed. Amsterdam, The Netherlands: Butterworth-Heinemann, 2002.

[24] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, “Digital twin in industry: State-of-the-art,” *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, Apr. 2019.

[25] A. Castellani, S. Schmitt, and S. Squartini, “Real-world anomaly detection by using digital twin systems and weakly supervised learning,” *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4733–4742, Jul. 2021.

[26] W. Ayadi *et al.*, “Leveraging lightweight AI for anomaly detection in mechanical power transmission at the edge,” *EPJ Web Conf.*, vol. 343, Art. no. 05014, 2025, doi: 10.1051/epjconf/202534305014.

[27] C. Banbury *et al.*, “MLPerf Tiny benchmark,” in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks*, 2021.

[28] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, San Diego, CA, USA, 2015.

[29] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.

[30] Toshiba Corporation, “TB6612FNG: Driver IC for Dual DC Motor,” datasheet. [Online]. Available: https://toshiba.semicon-storage.com

[31] Texas Instruments, “INA219 Zerø-Drift, Bidirectional Current/Power Monitor With I²C Interface,” datasheet SBOS448. [Online]. Available: https://www.ti.com

[32] InvenSense Inc., “MPU-6000 and MPU-6050 Product Specification,” Rev. 3.4, 2013.

[33] R. Barry, *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016.

[34] Espressif Systems, “ESP32-S3 Series Datasheet,” Shanghai, China. [Online]. Available: https://www.espressif.com

Appendix-1: Firmware Source Code (Edge-AI Modules)

The complete firmware, backend, dashboard and training code are in the project repository. The listings below are the neural-tuner inference, the adaptive control law and the anomaly classifier that run on the ESP32-S3; the identification module is described in Section 3.1.4.

A1.1 Neural PI tuner inference (nn_tuner.h)

```
#pragma once
// Neural PI tuner inference (weights from ml/train_tuner.py -> nn_tuner_weights.h).
// Input: identified model (K RPM/PWM, tau s, delay samples). Output: PI gains.
// MLP 3-16-16-2, tanh hidden layers, linear output = normalised [log Kp, log Ki].
#include <math.h>
#include "nn_tuner_weights.h"

inline void nnTunerPredict(float K, float tau, float delay, float &kp, float &ki) {
  float x[16], h1[16], h2[16];
  const float in[3] = {logf(K), logf(tau), delay};
  for (int i = 0; i < NN_SIZES[0]; i++) x[i] = (in[i] - NN_X_MEAN[i]) / NN_X_STD[i];

  for (int j = 0; j < NN_SIZES[1]; j++) {
    float z = NN_B0[j];
    for (int i = 0; i < NN_SIZES[0]; i++) z += x[i] * NN_W0[i * NN_SIZES[1] + j];
    h1[j] = tanhf(z);
  }
  for (int j = 0; j < NN_SIZES[2]; j++) {
    float z = NN_B1[j];
    for (int i = 0; i < NN_SIZES[1]; i++) z += h1[i] * NN_W1[i * NN_SIZES[2] + j];
    h2[j] = tanhf(z);
  }
  float out[2];
  for (int j = 0; j < NN_SIZES[3]; j++) {
    float z = NN_B2[j];
    for (int i = 0; i < NN_SIZES[2]; i++) z += h2[i] * NN_W2[i * NN_SIZES[3] + j];
    out[j] = z * NN_Y_STD[j] + NN_Y_MEAN[j];
  }
  kp = expf(out[0]);
  ki = expf(out[1]);
}
```

A1.2 Control law with adaptive gains (excerpt of main.cpp)

```
    // Control law
    float pwm = 0;
    bool cleanForId = true;
    float gKp = 0, gKi = 0, gKd = 0;
    switch (s.mode) {
      case Mode::PID:
      case Mode::Adaptive: {
        float kp = s.kp, ki = s.ki, kd = s.kd;
        // Adaptive AI: self-tuned PI from the on-device model, once the estimate is
trustworthy.
        // Adaptive is a PI controller: no D term at all. Derivative on the 10 ms speed
estimate feeds
        // measurement noise straight into the PWM, which also biases the closed-loop
identification
        // (it drove K negative with the dashboard's default Kd=0.05).
        if (s.mode == Mode::Adaptive) {
          kd = 0;
          if (est.confident) {
            kp = s.tunerNN ? est.nnKp : est.kp;
            ki = s.tunerNN ? est.nnKi : est.ki;
          }
        }
        // Bumpless transfer: when Ki changes (self-tuning, or new gains from the dashboard),
rescale
        // the integral so Ki*integral, i.e. the PWM it contributes, stays continuous.
        if (lastKi > 1e-6f && ki > 1e-6f && ki != lastKi) integral *= lastKi / ki;
        lastKi = ki;
        float dRpm = (rpm - lastRpmLocal) / dt;  // derivative on measurement: no kick on
setpoint steps
        float unsat = kp * error + ki * (integral + error * dt) - kd * dRpm;
```

```
        // Conditional integration: stop winding up while the output is saturated in that
direction.
        if (!((unsat > 255.0f && error > 0) || (unsat < 0.0f && error < 0)))
          integral = constrain(integral + error * dt, -INTEGRAL_LIMIT, INTEGRAL_LIMIT);
        pwm = kp * error + ki * integral - kd * dRpm;
        cleanForId = kd <= 0.0f;
        gKp = kp; gKi = ki; gKd = kd;
        break;
      }
      case Mode::Manual: pwm = s.targetRpm * 0.6f; break;
      case Mode::Raw:    pwm = s.rawPwm; break;
    }
    pwm = constrain(pwm, 0.0f, 255.0f);
```

A1.3 Anomaly classification (excerpt of anomaly.h)

```
  void classify() {
    const float zs = s_.zSpeed, zi = s_.zCurrent, zv = s_.zVib;
    // Severity: z-scores mapped so |z| = 3 is the edge of normal and |z| = 8 saturates.
    auto sev = [](float z) { return fminf(fmaxf((fabsf(z) - 3.0f) / 5.0f, 0.0f), 1.0f); };
    float drift = fminf(fmaxf((fabsf(s_.kDriftPct) - 10.0f) / 30.0f, 0.0f), 1.0f);
    float therm = fminf(fmaxf((s_.thermal - 0.6f) / 0.4f, 0.0f), 1.0f);
    float raw = fmaxf(fmaxf(sev(zs), sev(zv)), fmaxf(fmaxf(sev(zi) * 0.7f, drift), therm));
    s_.score += 0.02f * (raw - s_.score);  // ~0.5 s smoothing, consistent with the class
persistence

    Anomaly c;
    if (therm > 0.5f)                    c = Anomaly::Thermal;
    else if (zs < -4.0f && zi > 2.0f)    c = Anomaly::Load;       // slower than the twin,
more current
    else if (zs < -4.0f)                 c = Anomaly::DriveLoss;  // slower without extra
current
    else if (zs > 4.0f)                  c = Anomaly::Overspeed;  // faster than the twin:
load released
    else if (zv > 5.0f)                  c = Anomaly::Vibration;  // MPU readings are bursty:
higher bar
    else if (zi > 5.0f)                  c = Anomaly::Load;       // current up while speed
held
    else if (drift > 0.0f)               c = Anomaly::Drift;
    else                                 c = Anomaly::Normal;
    // Persistence: a new diagnosis must hold for 0.3 s (0.5 s to return to NORMAL) before it
is
    // reported, so a single transient (e.g. the vibration burst of a sudden speed change)
does
    // not flicker an alarm.
    if (c == s_.cls) { pending_ = c; pendingN_ = 0; return; }
    if (c != pending_) { pending_ = c; pendingN_ = 0; }
    // vibration faults (bearing wear, imbalance) are persistent: require 1 s
    uint32_t need = c == Anomaly::Normal ? 50u : c == Anomaly::Vibration ? 100u : 30u;
    if (++pendingN_ >= need) { s_.cls = c; pendingN_ = 0; }
  }
```